1. Problem Title & Link

Title: **LeetCode 621: Task Scheduler**

Link: https://leetcode.com/problems/task-scheduler/

2. Problem Statement (Short Summary)

You are given a list of tasks represented by capital letters.

Example:

tasks = ["A","A","A","B","B","B"]

Each task takes **1 unit of time** to complete.

However, there is a cooling period n between two **same tasks**.

Example:

A _ _ A

If n = 2, after executing task **A**, we must wait **2 units** before executing **A again**.

During idle time, we can execute **other tasks**.

Goal:

Return the **minimum time required to complete all tasks**.

3. Examples (Input → Output)

Example 1

Input

tasks = ["A","A","A","B","B","B"]

n = 2

Output

8

Explanation

One valid schedule:

A B idle A B idle A B

Example 2

Input

tasks = ["A","C","A","B","D","B"]

n = 1

Output

6

Explanation

Schedule:

A B C A D B

No idle needed.

Example 3

Input

tasks = ["A","A","A","B","B","B"]

n = 0

Output

6

Explanation

No cooling time → execute sequentially.

4. Constraints

$1 \le$ tasks.length $\le 10^4$

tasks[i] is uppercase English letter

$0 \le n \le 100$

Observations:

- Only **26 task types**
- Tasks may repeat
- Scheduling problem

5. Core Concept (Pattern / Topic)

Greedy + Max Heap

Key idea:

Always schedule the **task with highest remaining frequency first**.

6. Thought Process (Step-by-Step Explanation)

Step 1 — Understand the Problem

If a task appears frequently:

A A A A

We must spread them out.

Example with n = 2:

A _ _ A _ _ A _ _ A

Step 2 — Greedy Idea

At each step:

Choose the task with the **highest remaining frequency**.

Why?

Because delaying frequent tasks leads to more idle slots later.

Step 3 — Use Max Heap

Steps:

1. Count frequency of tasks
2. Put frequencies in a **max heap**
3. Execute tasks in cycles of size:

n + 1

Each cycle allows different tasks.

7. Visual / Intuition Diagram (ASCII)

Example

tasks = A A A B B B

n = 2

Cycle size

n + 1 = 3

Execution

Cycle 1 → A B idle

Cycle 2 → A B idle

Cycle 3 → A B

Total time

8

8. Pseudocode (Language Independent)

```
count frequency of tasks

add frequencies to max heap

time = 0

while heap not empty:

   temp list
   cycle = n + 1

   while cycle > 0 and heap not empty:
```

```
        task = pop max
        task--

        if task > 0:
            add to temp

        cycle--
        time++

    push temp tasks back to heap

    if heap empty:
        break

    time += remaining cycle

return time
```

9. Code Implementation

Python

```
import heapq
from collections import Counter

class Solution:
    def leastInterval(self, tasks, n):

        freq = Counter(tasks)

        maxHeap = [-f for f in freq.values()]
        heapq.heapify(maxHeap)

        time = 0

        while maxHeap:

            temp = []
            cycle = n + 1
```

```
        while cycle > 0 and maxHeap:

            count = heapq.heappop(maxHeap)
            count += 1

            if count != 0:
                temp.append(count)

            time += 1
            cycle -= 1

        for t in temp:
            heapq.heappush(maxHeap, t)

        if not maxHeap:
            break

        time += cycle

    return time
```

Java

```
class Solution {
  public int leastInterval(char[] tasks, int n) {

    int[] freq = new int[26];

    for(char t : tasks){
        freq[t-'A']++;
    }

    PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>((a,b) -> b-a);

    for(int f : freq){
        if(f > 0)
```

```
            maxHeap.add(f);
        }

        int time = 0;

        while(!maxHeap.isEmpty()){

            List<Integer> temp = new ArrayList<>();
            int cycle = n + 1;

            while(cycle > 0 && !maxHeap.isEmpty()){

                int count = maxHeap.poll();
                count--;

                if(count > 0)
                    temp.add(count);

                time++;
                cycle--;
            }

            for(int t : temp)
                maxHeap.add(t);

            if(maxHeap.isEmpty())
                break;

            time += cycle;
        }

        return time;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(N log 26)

Since only **26 task types**, heap operations are small.

Simplified:

O(N)

Space Complexity

O(1)

Only 26 task frequencies stored.

11. Common Mistakes / Edge Cases

1. Forgetting idle slots
2. Not reinserting tasks into heap
3. Miscalculating cycle length
4. Infinite loop if heap not updated

12. Detailed Dry Run

Example

tasks = [A,A,A,B,B,B]

n = 2

Frequency

A → 3

B → 3

Cycle size

3

Execution

Cycle	Tasks	Time
1	A B idle	3
2	A B idle	6
3	A B	8

Answer

8

13. Common Use Cases (Real-Life / Interview)

Task scheduling patterns appear in:

- CPU process scheduling

- Job queues
- Server request scheduling
- Manufacturing pipelines

14. Common Traps (Important!)

1. Not prioritizing high-frequency tasks
2. Ignoring idle times
3. Wrong cycle length
4. Not updating heap after execution

15. Builds To (Related LeetCode Problems)

Heap / Greedy progression

LC 347 Top K Frequent Elements

LC 621 Task Scheduler

LC 703 Kth Largest Element in Stream

LC 215 Kth Largest Element in Array

LC 295 Find Median From Data Stream

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Simulation	$O(n^2)$	inefficient
Heap + Greedy	O(n log 26)	optimal

17. Why This Solution Works

Executing the **most frequent task first** ensures that large gaps between identical tasks are filled by other tasks whenever possible.

This greedy strategy minimizes idle time.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Variable task durations
2. Multiple CPUs available
3. Weighted tasks
4. Dynamic cooling time